

RentIQ – Documentação de Implantação em Kubernetes

Este documento descreve (a) a aplicação e seus componentes, (b) o roteiro de testes utilizado para validar a implantação e (c) os artefatos Kubernetes empregados na implantação da aplicação via Minikube + Helm.

1. A Aplicação

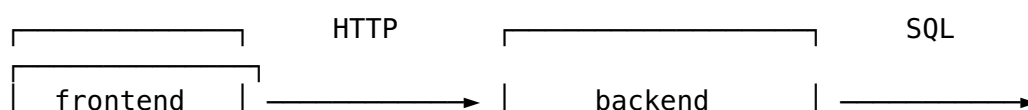
O **RentIQ** é uma aplicação web para predição de preços de aluguel de imóveis em São Carlos (SP), a partir de um modelo de machine learning (LightGBM) treinado sobre dados reais coletados via scraper. O usuário informa características do imóvel (tipo, bairro, área, quartos, banheiros, vagas) e recebe uma estimativa de aluguel com faixa de preço e os fatores que mais influenciaram o resultado.

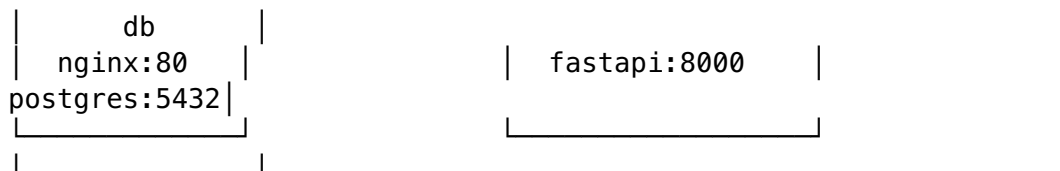
1.1 Stack tecnológica

Camada	Tecnologia
Frontend	React 19 + TypeScript + Vite
Backend	FastAPI + Uvicorn (Python 3.12)
Machine Learning	LightGBM, scikit-learn, pandas
Banco de dados	PostgreSQL 16
Servidor estático	Nginx
Contêineres	Docker
Orquestração	Kubernetes (Minikube) + Helm

1.2 Componentes / Containers

A aplicação é composta por três containers principais, que se comunicam conforme o diagrama abaixo:





- **frontend** — SPA em React compilada e servida via Nginx. Faz proxy reverso das chamadas `/api/*` para o backend, evitando expor a API diretamente ao navegador.
- **backend** — API REST em FastAPI. Carrega o modelo LightGBM serializado em disco e expõe os endpoints de autenticação, predição e histórico. Antes de subir, aguarda o banco de dados ficar disponível (via `initContainer`).
- **db** — Instância PostgreSQL 16, responsável por persistir usuários, bairros e histórico de predições.

1.3 Principais endpoints da API

Método	Rota	Auth	Descrição
GET	/health	—	Health check (usado pelos probes do K8s)
GET	/api/neighborhoods	—	Lista bairros disponíveis
GET	/api/model/metrics	—	Métricas do modelo (R^2 , MAE)
POST	/api/predictions	—	Gera predição de preço
GET	/api/predictions	JWT	Histórico de predições do usuário
POST	/api/auth/register	—	Cadastro de usuário (retorna JWT)
POST	/api/auth/login	—	Login de usuário (retorna JWT)
GET	/api/listings	JWT	Lista imóveis cadastrados pelo usuário
POST	/api/listings	JWT	Cadastra imóvel e retorna estimativa

2. Roteiro de Testes

Roteiro para validar, de ponta a ponta, que a implantação no Kubernetes (Minikube) está funcional. Assume-se que o ambiente já foi implantado seguindo o [README](#) (`helm upgrade --install rentiq ./helm/rentiq -n rentiq --create-namespace, minikube tunnel` e entrada em `/etc/hosts` configurados).

2.1 Verificação da infraestrutura

#	Passo	Comando	Resultado esperado
1	Cluster ativo	<code>minikube status</code>	Running
2	Pods no ar	<code>kubectl get pods -n rentiq</code>	3 pods (db, backend, frontend) em Running / 1/1 Ready
3	Services criados	<code>kubectl get svc -n rentiq</code>	Services db, backend, frontend presentes
4	Ingress criado	<code>kubectl get ingress -n rentiq</code>	rentiq-ingress com host <code>k8s.local</code>
5	Logs sem erro	<code>kubectl logs -f deployment/backend -n rentiq</code>	Sem stack traces; log de conexão bem-sucedida ao banco
6	Probes ok	<code>kubectl describe pod <backend-pod> -n rentiq</code>	Sem eventos de falha de readinessProbe/livenessProbe

2.2 Testes funcionais (via navegador / HTTP)

#	Cenário	Passos	Resultado esperado
1	Health check	<code>curl http://k8s.local/api/health</code> (ou <code>/health</code> conforme rota exposta)	HTTP 200
2			Página inicial do RentIQ carrega

#	Cenário	Passos	Resultado esperado
	Acesso ao frontend	Abrir <code>http://k8s.local</code> no navegador	
3	Listagem de bairros	GET <code>http://k8s.local/api/neighborhoods</code>	HTTP 200 com lista de bairros
4	Métricas do modelo	GET <code>http://k8s.local/api/model/metrics</code>	HTTP 200 com $R^2 \approx 0,70$ e MAE $\approx R\$ 514$
5	Login com usuário demo	Login com <code>demo@rentiq.com / rentiq123</code> na tela de login	Autenticação bem-sucedida, JWT retornado
6	Predição sem autenticação	Preencher formulário de predição (tipo, bairro, área, quartos, banheiros, vagas) e enviar	Retorna preço estimado com faixa mín/máx e fatores de influência
7	Histórico de predições	Logado, acessar histórico de predições	Lista as predições realizadas pelo usuário
8	Cadastro de imóvel	Logado, cadastrar um imóvel (POST <code>/api/listings</code>)	Imóvel criado e estimativa retornada
9	Persistência do banco	Reiniciar o pod db (<code>kubectl delete pod <db-pod></code>)	Novo pod sobe automaticamente (o Deployment recria); dados são perdidos (armazenamento <code>emptyDir</code>), comportamento esperado neste ambiente de demonstração

2.3 Testes de resiliência / auto-recuperação

#	Cenário	Passos	Resultado esperado
1	Crash do backend	<code>kubectl delete pod <backend></code>	Kubernetes recria o pod automaticamente; <code>initContainer</code> aguarda o banco antes de subir a API

#	Cenário	Passos	Resultado esperado
2	Rollback de release	pod> -n rentiq helm rollback rentiq <revisão- anterior> -n rentiq	Release volta ao estado anterior sem erros
		Rodar helm upgrade -- install novamente sem alterações	
3	Atualização idempotente		Nenhum recurso é recriado desnecessariamente (helm status mostra “up-to-date”)

2.4 Critérios de aceite

- Todos os pods em estado Running/Ready.
- Aplicação acessível via `http://k8s.local`.
- Fluxo completo (login → predição → histórico) funcional sem erros 4xx/5xx inesperados.
- Nenhum CrashLoopBackOff ou ImagePullBackOff durante o ciclo de vida dos pods.

3. Artefatos Kubernetes

A implantação é feita via **Helm Chart** localizado em `helm/rentiq`, versão `0.1.0` (`appVersion: 1.0.0`). Os valores parametrizáveis (imagens, réplicas, recursos, credenciais, timings de probe, host do Ingress) estão centralizados em `values.yaml`.

3.1 Resumo dos objetos

Recurso	Objeto Kubernetes	Descrição
db	Deployment + Service	PostgreSQL 16, armazenamento efêmero (<code>emptyDir</code>) — dados resetam a cada restart do pod
db-secret	Secret	

Recurso	Objeto Kubernetes	Descrição
		Usuário, senha e nome do banco (POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB)
backend	Deployment + Service	API FastAPI, com initContainer que aguarda o banco subir; probes usam GET /health
backend-secret	Secret	DATABASE_URL e JWT_SECRET
backend-config	ConfigMap	ALLOWED_ORIGINS, MODEL_PATH, STAGE, DEBUG
frontend	Deployment + Service	SPA React servida pelo Nginx, com proxy reverso de /api/ para o backend
rentiq-ingress	Ingress	Publica o frontend em http://k8s.local

3.2 Deployment — db

- Imagem: postgres:16-alpine.
- Estratégia de rollout: Recreate (evita dois pods de Postgres escrevendo simultaneamente no mesmo volume).
- Volume: emptyDir montado em /var/lib/postgresql/data (armazenamento efêmero — adequado para ambiente de demonstração, não para produção).
- Variáveis de ambiente injetadas via Secret db-secret.
- readinessProbe/livenessProbe: pg_isready -U <user> (delay inicial de 5s/15s, período de 5s/10s).
- Requests/Limits: 100m–500m CPU, 128Mi–512Mi memória.

3.3 Deployment — backend

- Imagem: josemaia123/rentiq-backend:latest (parametrizável via --set backend.image.repository/tag).
- initContainer (wait-for-db): executa pg_isready em loop contra o Service db até o banco responder, garantindo ordem de subida correta.
- Variáveis de ambiente: envFrom combinando ConfigMap (backend-config) e Secret (backend-secret).
- readinessProbe/livenessProbe: GET /health na porta 8000 (delay 10s/20s, período 10s/20s).
- Requests/Limits: 200m–1 CPU, 256Mi–1Gi memória.
- Réplicas: 1 (replicaCount, configurável).

3.4 Deployment — frontend

- Imagem: josemaia123/rentiq-frontend:latest (parametrizável via `--set frontend.image.repository/tag`).
- Serve a SPA React compilada via Nginx na porta 80, com proxy reverso de `/api/` para o Service backend.
- `readinessProbe/livenessProbe`: GET / na porta 80 (delay 5s/10s, período 10s/20s).
- Requests/Limits: 50m–250m CPU, 64Mi–256Mi memória.
- Réplicas: 1 (`replicaCount`, configurável).

3.5 Services

Todos do tipo ClusterIP (padrão), expondo cada Deployment internamente ao cluster via selector: `app: <nome>`:

- db → porta 5432.
- backend → porta 8000.
- frontend → porta 80.

3.6 ConfigMap — backend-config

Armazena configurações não sensíveis do backend:

```
ALLOWED_ORIGINS: '["http://k8s.local"]'  
MODEL_PATH: "models/modelo_aluguel.pkl"  
STAGE: "production"  
DEBUG: "false"
```

3.7 Secrets

- **db-secret**: POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB.
- **backend-secret**: JWT_SECRET e DATABASE_URL (montado dinamicamente a partir dos valores de `db.user`, `db.password`, `db.name` e `db.port`, apontando para o Service db).

Observação: no chart atual, os valores de `values.yaml` (senha do banco, `JWT_SECRET`) são texto plano versionado no repositório — adequado para o ambiente de demonstração local (Minikube), mas deveria ser substituído por um cofre de segredos (Sealed Secrets, Vault, etc.) em um cenário produtivo real.

3.8 Ingress — rentiq-ingress

- `ingressClassName`: nginx (addon ingress do Minikube).

- Regra de host: `k8s.local` → todo tráfego (path: `/`, pathType: `Prefix`) é roteado para o Service frontend, porta 80.
- Requer `minikube addons enable ingress` e `minikube tunnel` para expor o LoadBalancer/Ingress em `127.0.0.1`, além da entrada `127.0.0.1 k8s.local` no `/etc/hosts`.

3.9 Fluxo de implantação (resumo)

1. `minikube start + minikube addons enable ingress`.
2. Build das imagens de backend e frontend (local, via `minikube docker-env`, ou publicadas no Docker Hub).
3. `helm upgrade --install rentiq ./helm/rentiq -n rentiq --create-namespace` — cria o namespace `rentiq` (se não existir) e aplica todos os objetos acima.
4. `kubectl get pods,svc,ingress -n rentiq` para acompanhar o rollout.
5. `minikube tunnel` + entrada em `/etc/hosts` para acessar via `http://k8s.local`.

Esse fluxo completo está automatizado nos targets `make k8s-up` / `make k8s-up-pull` do [Makefile](#); o passo a passo manual equivalente está documentado na Opção 1 do [README](#).